A Universal Dynamic Programming Policy for the Door-Key Navigation Problem

Shahid Mohammad Mulla
Department of Electrical and Computer Engineering
University of California, San Diego
Student ID: A69043383
smulla@ucsd.edu

***Abstract*—This report presents a Dynamic Programming solution to the Door-Key navigation problem, formulated as a deterministic Markov Decision Process. Two scenarios are addressed: a *Known Map* setting, in which an optimal policy is computed for each of seven provided environments, and a *Random Map* setting, in which a single universal policy is computed once over a 16,704-state space and applied to all 36 randomly configured 10×10 environments without recomputation. Synchronous value iteration (backward induction) with strictly positive stage costs guarantees convergence to the globally optimal cost-to-go. The algorithm converges in 12–25 iterations across all Known Map environments and in 25 iterations for the Random Map, yielding optimal trajectories with total costs ranging from 13 to 54 for Part A and 14 to 53 for Part B.**

***Index Terms*—Dynamic programming, Markov decision process, value iteration, gridworld planning.**

I. Introduction

Motion planning under environmental constraints is a core challenge in robotics. Real-world tasks often require an agent to interact with its environment in a prescribed order; retrieving a tool before using it, or opening a passage before traversing it, before reaching a goal. The Door-Key problem captures this structure in a discrete gridworld: an agent must navigate to a goal cell, picking up a key and unlocking a door along the way if required. Despite its apparent simplicity, the problem demands reasoning over a composite state that couples pose, object possession, and door status, making it a useful benchmark for sequential decision-making algorithms.

The problem is directly relevant to robotics scenarios such as autonomous indoor navigation, where a robot may need to retrieve an access card before passing through a secured door, or manipulation planning, where a tool must be acquired before a task can be completed. In both cases the planner must account for how the agent's actions change not only its location but also the state of objects in the environment.

We formulate the problem as a deterministic MDP and solve it via backward induction (synchronous value iteration). Two variants are considered: a *Known Map* setting with seven fixed environments solved individually, and a *Random Map* setting with 36 configurations solved by a single universal policy whose state space encodes the variable map parameters. The policy is extracted greedily from the converged value function and is guaranteed to be globally optimal under the given stage costs.

II. Environment

A. Gridworld Layout

The Door-Key environment is a discrete gridworld implemented via `gym-minigrid`. The grid is bounded by impassable walls; interior wall cells partition the space into regions connected by doors. The agent (red triangle) must reach a goal cell (green square), collecting a key (yellow object) and unlocking any locked doors along the way. Black cells are freely traversable; grey cells are obstacles. A representative environment is shown in Fig. 1.

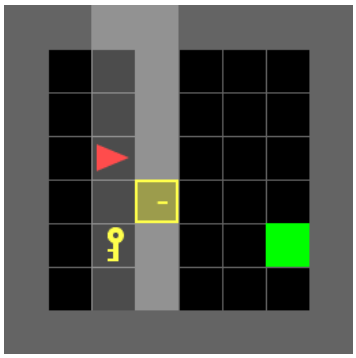


Fig. 1: A Door-Key environment. The agent (red triangle) must reach the goal (green square), collecting the key (yellow) to unlock the door if required.

B. Actions and Costs

The agent has five actions: Move Forward (MF), Turn Left (TL), Turn Right (TR), Pick Up Key (PK), and Unlock Door (UD). Every action incurs a strictly positive cost regardless of whether it has a physical effect — an invalid action (e.g. PK when the key is not directly in front) still charges its full cost. Action costs are listed in Table I.

TABLE I: Action costs.

Action	Notation	Cost
Turn Left	TL	1
Turn Right	TR	1
Move Forward	MF	3
Pick Up Key	PK	2
Unlock Door	UD	5

III. Problem Formulation

The Door-Key navigation task is formulated as a deterministic, undiscounted MDP defined by the tuple $(\mathcal{X}, \mathcal{U}, f, \ell, q, T)$. The elements common to both scenarios are defined below; the state space and initial condition, which differ between scenarios, are treated in the subsections that follow.

Control Space

The control space is identical for both scenarios:

$$\mathcal{U} = \{\text{MF, TL, TR, PK, UD}\} \tag{1}$$

with integer encoding $\{0, 1, 2, 3, 4\}$ respectively.

Stage Cost and Terminal Cost

The stage cost $\ell : \mathcal{X} \times \mathcal{U} \to \mathbb{R}_{>0}$ depends only on the action taken:

$$\ell(x, u) = \begin{cases} 1 & u \in \{\text{TL, TR}\} \\ 3 & u = \text{MF} \\ 2 & u = \text{PK} \\ 5 & u = \text{UD} \end{cases} \tag{2}$$

All costs are strictly positive, including for actions with no physical effect (e.g. PK when the key is not directly in front). The terminal cost is:

$$q(x) = \begin{cases} 0 & (c_x, c_y) = \mathbf{g} \\ \infty & \text{otherwise} \end{cases} \tag{3}$$

where $\mathbf{g}$ is the goal cell.

Planning Horizon and Discount

The problem is undiscounted ($\gamma = 1$) with an effectively infinite planning horizon. Since all stage costs are strictly positive and the goal is an absorbing state with zero cost, the optimal cost-to-go is finite and the horizon need not be specified explicitly, therefore, the DP is run until convergence.

Motion Model

The transition function $f : \mathcal{X} \times \mathcal{U} \to \mathcal{X}$ is deterministic. Let (c_x, c_y, θ) denote the agent's column, row, and heading, where $\theta \in \{0, 1, 2, 3\}$ encodes the four cardinal directions: Right (R), Down (D), Left (L), and Up (U). Let $\boldsymbol{\delta}(\theta)$ be the unit displacement vector for direction θ, with $\boldsymbol{\delta} \in \{(1, 0), (0, 1), (-1, 0), (0, -1)\}$ for $\theta \in \{\text{R, D, L, U}\}$. The front cell is $\mathbf{p}_f = (c_x, c_y) + \boldsymbol{\delta}(\theta)$. The transitions are:

$$\text{TL} : \theta \leftarrow (\theta - 1) \bmod 4 \tag{4}$$

$$\text{TR} : \theta \leftarrow (\theta + 1) \bmod 4 \tag{5}$$

$$\text{MF} : (c_x, c_y) \leftarrow \mathbf{p}_f \quad \text{if } \mathbf{p}_f \notin \mathcal{W} \text{ and } \mathbf{p}_f \text{ is not a locked door} \tag{6}$$

$$\text{PK} : k \leftarrow 1 \quad \text{if } \mathbf{p}_f = \mathbf{p}_{\text{key}} \text{ and } k = 0 \tag{7}$$

$$\text{UD} : d \leftarrow 1 \quad \text{if } \mathbf{p}_f = \mathbf{p}_{\text{door}},\ k = 1, \text{ and } d = 0 \tag{8}$$

where $\mathcal{W}$ is the set of wall cells. If the stated condition is not met, the action is a *no-operation* (no-op): the state remains unchanged, but the stage cost is still incurred in full.

A. *Part A: Known Map*

State Space: For a known environment with a single door, the state is:

$$x = (c_x,\ c_y,\ \theta,\ k,\ d) \ \in \ \mathcal{X}_A \tag{9}$$

where $c_x \in \{0, \ldots, W-1\}$, $c_y \in \{0, \ldots, H-1\}$ are the agent's column and row in a grid of width W and height H; $\theta \in \{0, 1, 2, 3\}$ encodes the heading (R, D, L, U); $k \in \{0, 1\}$ indicates whether the agent carries the key; and $d \in \{0, 1\}$ indicates whether the door is open. Wall cells are excluded: $(c_x, c_y) \notin \mathcal{W}$.

Initial State: The initial state $x_0 = (c_x^0,\ c_y^0,\ \theta^0,\ 0,\ 0)$ is read directly from the environment: the agent starts without the key and with the door locked.

B. *Part B: Random Map*

State Space: The random map has a fixed 10×10 grid with two doors at $\mathbf{p}_{d_1} = (5, 3)$ and $\mathbf{p}_{d_2} = (5, 7)$, three possible key positions $\mathcal{K} = \{(2, 2), (2, 3), (1, 6)\}$ indexed by $\kappa \in \{0, 1, 2\}$, and three possible goal positions $\mathcal{G} = \{(6, 1), (7, 3), (6, 6)\}$ indexed by $\phi \in \{0, 1, 2\}$. To construct a *single* policy valid for all 36 environment configurations, the key and goal indices are embedded in the state:

$$x = (c_x,\ c_y,\ \theta,\ k,\ d_1,\ d_2,\ \kappa,\ \phi) \ \in \ \mathcal{X}_B \tag{10}$$

where $d_1, d_2 \in \{0, 1\}$ are the open/locked states of each door, $\kappa \in \{0, 1, 2\}$ is the key position index, and $\phi \in \{0, 1, 2\}$ is the goal position index. The indices κ and ϕ are static, they do not change under any action, but conditioning the policy on them allows a single value function to encode optimal behaviour for every configuration simultaneously. The total state space has $|\mathcal{X}_B| = 16{,}704$ states.

Initial State: The agent is always spawned at $(4, 8)$ facing up, without the key:

$$x_0 = (4,\ 8,\ \text{UP},\ 0,\ d_1^0,\ d_2^0,\ \kappa,\ \phi) \tag{11}$$

where $d_1^0, d_2^0 \in \{0, 1\}$ are read from the loaded environment and κ, ϕ are resolved by matching the environment's key and goal positions against $\mathcal{K}$ and $\mathcal{G}$.

C. *Objective*

A policy is a mapping $\pi : \mathcal{X} \to \mathcal{U}$. The objective is to find the policy π^* that minimises the total cost-to-go from every state:

$$V^*(x) = \min_{\pi} \sum_{t=0}^{T} \ell\big(x_t,\ \pi(x_t)\big) + q(x_T), \quad x_0 = x \tag{12}$$

where the trajectory $\{x_t\}$ evolves under $x_{t+1} = f(x_t, \pi(x_t))$. The optimal value function satisfies the DP recursion:

$$V_{k+1}(x) = \min_{u \in \mathcal{U}} \Big[\ell(x, u) + V_k\big(f(x, u)\big)\Big] \tag{13}$$

with boundary condition $V_0(x) = q(x)$ for all $x \in \mathcal{X}$. The optimal policy is recovered as:

$$\pi^*(x) = \arg\min_{u \in \mathcal{U}} \Big[\ell(x, u) + V_k\big(f(x, u)\big)\Big] \tag{14}$$

IV. TECHNICAL APPROACH

The objective defined in Section III is solved via synchronous value iteration, applied over the full state space $\mathcal{X}$ without a fixed horizon. The algorithm is identical in structure for both scenarios; the only difference is the state space over which it operates. Algorithm 1 gives the complete procedure.

A. State Space Enumeration

All valid states are enumerated explicitly before the iteration begins. For Part A, a state (c_x, c_y, θ, k, d) is valid if and only if $(c_x, c_y) \notin \mathcal{W}$, where $\mathcal{W}$ is obtained by scanning the loaded environment for `Wall` objects. For Part B, validity additionally requires (c_x, c_y) to lie within the 10×10 grid interior, with the fixed wall geometry described in Section III applied. The enumeration visits all combinations of position, heading, and discrete flags, yielding $|\mathcal{X}_A|$ states per known map and $|\mathcal{X}_B| = 16{,}704$ states for the random map.

B. Initialization

The value function is initialised from the terminal cost q:

$$V_0(x) = \begin{cases} 0 & (c_x, c_y) = \mathbf{g}(\phi) \\ \infty & \text{otherwise} \end{cases} \quad \forall\, x \in \mathcal{X} \tag{15}$$

where $\mathbf{g}(\phi)$ is the goal cell (dependent on ϕ in Part B, fixed in Part A). Goal states are absorbing: the policy is undefined there and their value is never updated. The ∞ initialisation encodes the fact that no path to the goal has been discovered yet. So, the states acquire finite values only as the cost-to-go propagates backward from the goal.

C. Synchronous Value Iteration

At each iteration k, every non-goal state is updated simultaneously using V_k:

$$V_{k+1}(x) = \min_{u \in \mathcal{U}} \underbrace{\Big[\ell(x,\, u) + V_k\big(f(x,u)\big)\Big]}_{Q_k(x,\,u)}, \quad \forall\, x \in \mathcal{X} \tag{16}$$

The term $Q_k(x,u)$ is the cost of taking action u in state x and then acting optimally thereafter according to the current estimate V_k. The minimisation selects the action that balances immediate cost $\ell(x,u)$ against the cost-to-go from the resulting state $f(x,u)$.

The optimal policy is recorded alongside the value update:

$$\pi^*(x) = \arg\min_{u \in \mathcal{U}} \Big[\ell(x,\, u) + V_k\big(f(x,u)\big)\Big] \tag{17}$$

Because the update is *synchronous*, all states read from V_k and write to a fresh V_{k+1} in the same sweep, therefore no state observes a partially updated value function within a single iteration.

D. Transition Evaluation

At each value update, $f(x,u)$ is evaluated by applying the transition rules from Section III directly in code, no environment step is called. Each of the five actions resolves to one of three outcomes: a change in heading (TL, TR), a change in position (MF, if unblocked), or a change in a discrete flag (PK sets $k \leftarrow 1$; UD sets $d \leftarrow 1$). If the precondition for an action is not met, $f(x,u) = x$ and the update becomes $\ell(x,u) + V_k(x)$, meaning the full action cost is charged with no progress. This action is never preferred over a productive one unless all actions lead to states with $V_k = \infty$.

E. Convergence

Guarantee: Convergence is guaranteed by two properties of this MDP. First, all stage costs are strictly positive: $\ell(x,u) \geq 1$ for all (x,u). Second, the goal is absorbing with $q = 0$. Together, these imply that $V_k(x)$ is monotonically non-increasing and bounded below by zero for every state reachable from the goal, so $V_k \to V^*$ as $k \to \infty$.

Termination Criterion: The iteration halts when the maximum change in the value function satisfies:

$$\Delta_k = \max_{\substack{x \in \mathcal{X} \\ V_{k+1}(x) < \infty}} |V_{k+1}(x) - V_k(x)| < \varepsilon, \quad \varepsilon = 10^{-6} \tag{18}$$

The maximum is restricted to states with $V_{k+1}(x) < \infty$. States that remain ∞ in both V_k and V_{k+1} contribute a delta of zero by definition and are excluded; including them would cause Δ_k to read zero in the first iteration before any finite values have propagated, triggering a false convergence. States with $V_k = \infty$ but $V_{k+1} < \infty$ contribute $\Delta_k = V_{k+1}(x)$ (the full new value), correctly signalling that the iteration has not yet settled.

F. Policy Extraction

Once V^* has converged, the optimal action sequence from any initial state x_0 is obtained by greedy rollout under the frozen policy π^*:

$$x_{t+1} = f\big(x_t,\, \pi^*(x_t)\big), \quad \text{until } (c_x^t, c_y^t) = \mathbf{g} \tag{19}$$

The total cost of the extracted sequence equals $V^*(x_0)$ exactly, since each step applies the action that attains the minimum in Eq. 17. Termination is guaranteed for any state with $V^*(x_0) < \infty$; a hard cap of 500 steps is enforced in the implementation as a safety guard against floating-point edge cases.

G. Part B: Universal Policy via State Augmentation

For the random map, a single DP solve must cover all 36 environment configurations: 3 key positions $\times$ 3 goal positions $\times$ 2^2 door state combinations. Rather than solving 36 separate MDPs, the key index κ and goal index ϕ are embedded as static components of the state (Section III). Since f never modifies κ or ϕ, these indices propagate unchanged through every transition, but they condition the goal test in $\pi^*(x)$ and the key position lookup in $f(\cdot, \mathrm{PK})$ and $f(\cdot, \mathrm{MF})$.

The recursion in Eq. 16 is therefore applied once over $\mathcal{X}_B$, simultaneously solving all 36 sub-problems. The cost-to-go

for configuration $(\kappa, \phi, d_1^0, d_2^0)$ starting at $(4, 8, \text{UP})$ is simply $V^*(4, 8, \text{UP}, 0, d_1^0, d_2^0, \kappa, \phi)$, read directly from the converged table with no recomputation.

At runtime, `build_init_state` constructs the 8-tuple initial state by matching the loaded environment's key and goal positions against the fixed index sets $\mathcal{K}$ and $\mathcal{G}$, and reading door states directly from the environment object. The precomputed policy is then queried for every step of the rollout.

Algorithm 1 Synchronous Value Iteration for Door-Key MDP

Input: State space $\mathcal{X}$, transition function f, stage cost ℓ, goal set $\mathcal{G}$, threshold ε.
Output: Optimal value function V^*, optimal policy π^*.

Initialise:
  $V_0(x) \leftarrow 0 \ \forall x : (c_x, c_y) \in \mathcal{G}$
  $V_0(x) \leftarrow \infty \ \forall x : (c_x, c_y) \notin \mathcal{G}$
$k \leftarrow 0$
repeat
  for each $x \in \mathcal{X}$ with $(c_x, c_y) \notin \mathcal{G}$ **do**
    for each $u \in \mathcal{U}$ **do**
      $Q_k(x, u) \leftarrow \ell(x, u) + V_k\big(f(x, u)\big)$
    end for
    $V_{k+1}(x) \leftarrow \min_u Q_k(x, u)$
    $\pi^*(x) \leftarrow \arg\min_u Q_k(x, u)$
  end for
  $\Delta_k \leftarrow \max\big\{|V_{k+1}(x) - V_k(x)| : V_{k+1}(x) < \infty\big\}$
  $k \leftarrow k + 1$
until $\Delta_{k-1} < \varepsilon$
$V^* \leftarrow V_k$

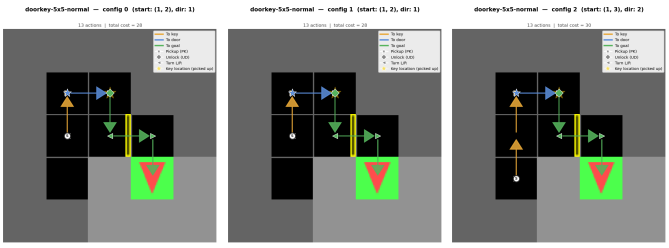
Policy rollout from x_0**:**
$t \leftarrow 0$, sequence $\leftarrow [\,]$
while $(c_x^t, c_y^t) \notin \mathcal{G}$ **do**
  $u_t \leftarrow \pi^*(x_t)$
  Append u_t to sequence
  $x_{t+1} \leftarrow f(x_t, u_t)$; $t \leftarrow t + 1$
end while
return V^*, π^*, sequence

V. Results & Discussion

Animated rollouts (GIFs) for all environments are available at this Drive link. In all trajectory images: arrows indicate MF steps (coloured by phase), ⋆ marks PK, ◊ marks UD, </> mark turns, and **S** marks the start. Asfor the colour of the arrows: Orange = heading to key, blue = heading to door, green = heading to goal.

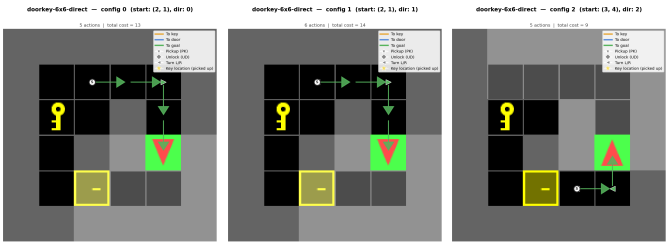
Quantitative Summary: Table II reports the DP convergence and optimal cost for each map's canonical starting configuration (cfg0).

Trajectory Visualisations: Figs. 2–8 show all 21 trajectories. Each row shows the three configurations of one environment: cfg0 is the canonical start from the environment file; cfg1 and cfg2 are alternative start positions sampled from the converged value function, chosen at the 1/3 and 2/3 marks of all reachable states for spatial spread. The policy is computed once per map and all three rollouts query the same precomputed V^* and π^* without recomputation.



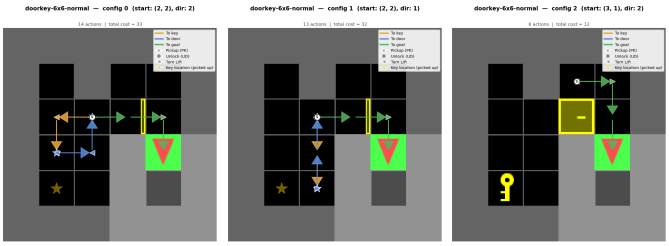
(a) cfg0 — cost 28 (b) cfg1 — cost 28 (c) cfg2 — cost 30

Fig. 2: **doorkey-5x5-normal.** cfg0 and cfg1 both cost 28, as their starts lie on the same shortest path via key and door. cfg2 costs 30 from a less favourable start, but the full key-pickup, door-unlock, and goal sequence is visible in the phase-coloured trajectory (orange → blue → green).



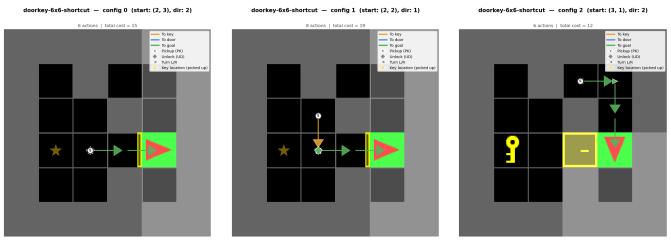
(a) cfg0 — cost13 (b) cfg1 — cost 14 (c) cfg2 — cost 9

Fig. 3: **doorkey-6x6-direct.** With the door adjacent to the canonical start, the detour in the normal layout is removed. cfg0 and cfg1 reach the goal with costs 13 and 14 (lowest for 6×6 starts), differing by one turn due to initial heading in cfg1. cfg2 costs 9, starting on the goal side with no key or unlock required.



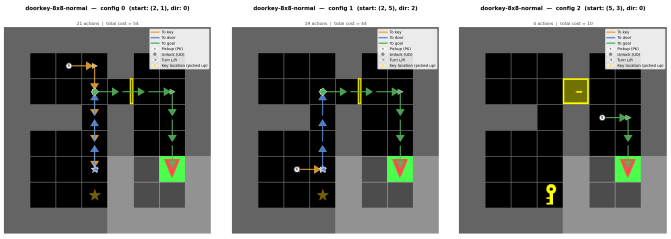
(a) cfg0 — cost 33 (b) cfg1 — cost 32 (c) cfg2 — cost 12

Fig. 4: **doorkey-6x6-normal.**The full three-phase trajectory (orange → blue → green) is visible in both cfg0 and cfg1, with the agent navigating across the grid to collect the key before unlocking the door. cfg2 starts close to the goal, yielding cost 12 via a short 6-action path.



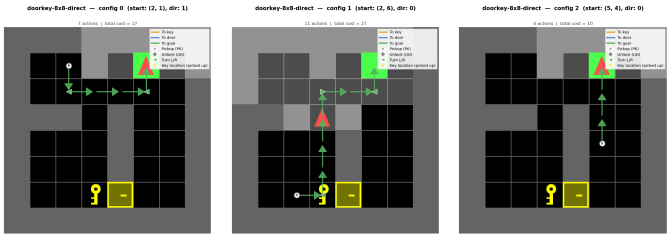
(a) cfg0 — cost 15 (b) cfg1 — cost 19 (c) cfg2 — cost 12

Fig. 5: **doorkey-6x6-shortcut.** cfg0 (cost 15) routes through an already-open door directly to the goal, no key interaction required. cfg1 (cost 19) must pick up the key and unlock the door, but the key is conveniently placed close to the agent's start, keeping the additional cost low. cfg2 (cost 12) begins on the same side as the goal, bypassing the door entirely.



(a) cfg0 — cost 54 (b) cfg1 — cost 44 (c) cfg2 — cost 10

Fig. 6: **doorkey-8x8-normal.** The largest grid with the worst-case layout produces the highest cost across all Part A environments (54 for cfg0, 25 DP iterations to converge). The full three-phase trajectory is clearly visible in cfg0. cfg2 starts near the goal and costs only 10, demonstrating the policy's ability to exploit proximity.



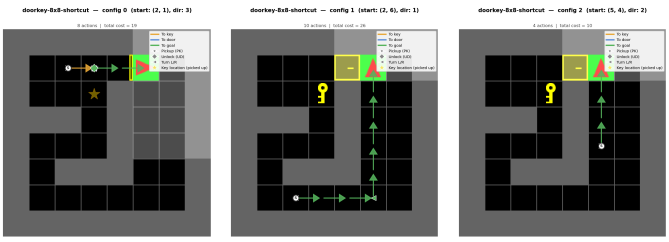
(a) cfg0 — cost 17 (b) cfg1 — cost 27 (c) cfg2 — cost 10

Fig. 7: **doorkey-8x8-direct.** Direct door placement reduces the canonical cost from 54 to 17 relative to the normal layout, a 69% reduction. cfg1 costs 27 because it starts farther from the door despite the favourable geometry.

Discussion: Three structural observations follow from Table II and Figs. 2–8.

Layout geometry directly determines cost. For 8×8 maps, the canonical cost drops from 54 (normal) to 19 (shortcut) to 17 (direct) as the door moves closer to the agent's start. The DP policy correctly reflects this: it routes through the shortcut only when doing so reduces total cost, and ignores it otherwise.

DP iterations scale with map diameter. Convergence



(a) cfg0 — cost 19 (b) cfg1 — cost 26 (c) cfg2 — cost 10

Fig. 8: **doorkey-8x8-shortcut.** The shortcut passage reduces the canonical cost to 19, between the normal (54) and direct (17) variants. The policy uses the shortcut only when it is on the cost-minimal path, bypassing it when a more direct route to the door is available.

TABLE II: Part A results for cfg0 (canonical start).

Map	DP iters	Actions	Cost
doorkey-5x5-normal	15	13	28
doorkey-6x6-normal	16	14	33
doorkey-6x6-direct	13	5	13
doorkey-6x6-shortcut	12	6	15
doorkey-8x8-normal	25	21	54
doorkey-8x8-direct	15	7	17
doorkey-8x8-shortcut	14	8	19

requires more iterations for larger grids (25 for 8×8-normal vs. 12 for 6×6-shortcut) because the value must propagate from the goal across more states before settling. Each iteration extends the finite-cost frontier by at most one step.

Alternate starts are handled without recomputation. The value function covers the full state space $\mathcal{X}_A$, so cfg1 and cfg2 rollouts are simply greedy reads from the same V^* computed for cfg0. The cost variation across configurations (e.g. 28, 28, 30 for 5×5-normal) reflects true geometric distance from each start to the goal under the given action costs.

A. *Part B: Random Map*

Quantitative Summary: The universal policy was computed in a single DP solve over $|\mathcal{X}_B| = 16{,}704$ states, converging in 25 iterations (max $\Delta_{20} = 55.0$, $\Delta_{25} < 10^{-6}$). It was then applied to all 36 environments with no recomputation. Costs range from 14 (environments where the goal is reachable without any door interaction) to 53 (key pickup required plus one or both doors locked, with the goal at the farthest position). This single solve is equivalent to running 36 separate Part A-style DPs, one for each $(\kappa, \phi, d_1^0, d_2^0)$ combination.

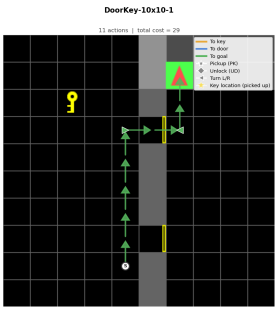
Trajectory Visualisations: Figs. 9–17 show all 36 trajectories grouped in sets of 4 (numerical order). Each group of four shares the same key and goal positions but varies in door states, directly illustrating how the universal policy adapts its path based on which doors are locked.

Discussion: **Universal policy correctness.** The policy succeeds on all 36 environments without a single failure. This confirms that embedding κ and ϕ in the state is both necessary and sufficient: the DP correctly resolves the trade-off between different key positions, goal locations, and door states in a single solve.

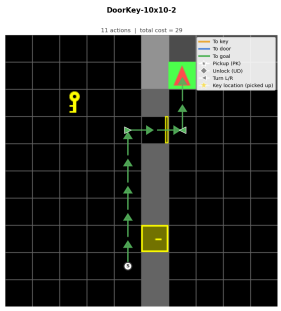
Cost drivers. The minimum observed cost is 14, achieved when both relevant doors are open and the goal is on the directly reachable side of the wall. The maximum is 53, requiring key pickup (cost 2), at least one UD action (cost 5), multiple MF steps (cost 3 each), and navigation to the farthest goal position. The UD action's cost of 5 is the single largest contributor to cost variance across configurations.

State space efficiency. Solving the 16,704-state extended MDP once is equivalent to 36 independent known-map solves. For comparison, a 10×10 grid with a single door has approximately $58 \times 4 \times 2 \times 2 = 928$ states; scaling to 36 configurations via state augmentation adds a factor of 18 (9 key-goal combinations $\times$ 2 per door, collapsed into κ, ϕ, d_1, d_2), yielding $928 \times 18 \approx 16{,}704$ — consistent with the enumerated count.

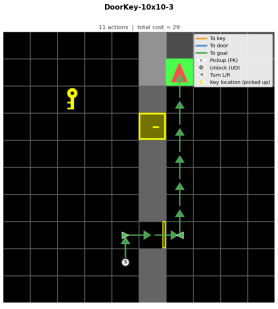
Convergence speed. The value function required 25 iterations to converge for Part B, with $\Delta_{20} = 55.0$ indicating that large cost updates were still propagating at iteration 20. The drop to $\Delta < 10^{-6}$ by iteration 25 reflects the bounded diameter of the 10×10 grid: once the finite-cost frontier reaches all reachable states, residual updates decay to zero in a small number of additional sweeps.



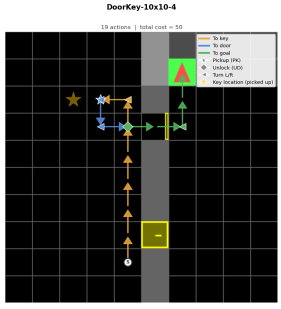
(a) Env 1 — cost 29



(b) Env 2 — cost 29

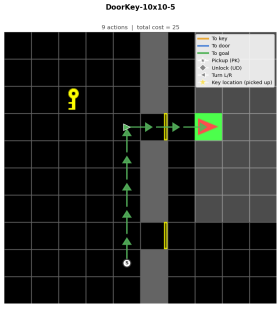


(c) Env 3 — cost 29

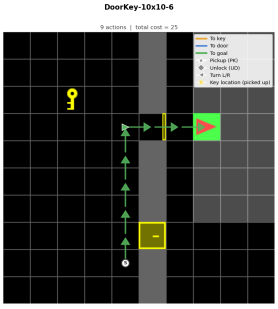


(d) Env 4 — cost 50

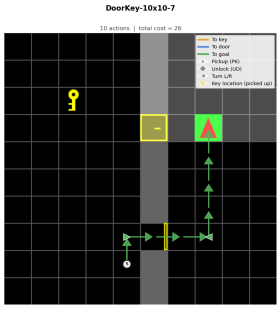
Fig. 9: Environments 1–4. All four share the same key and goal positions, differing only in door states. Env 1 has both doors open and navigates directly to the goal (cost 29). Envs 2 and 3 each have one door open, the agent passes through the open door, also achieving cost 29. Env 4 has both doors locked, requiring key pickup and an UD action before proceeding, which drives the cost up to 50.



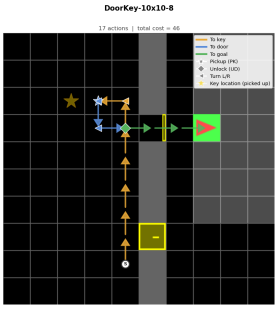
(a) Env 5 — cost 25



(b) Env 6 — cost 25

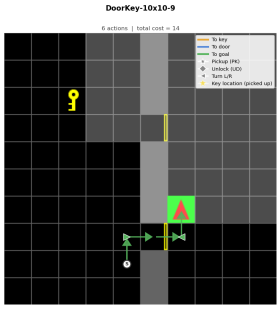


(c) Env 7 — cost 26

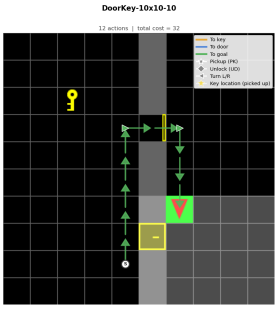


(d) Env 8 — cost 46

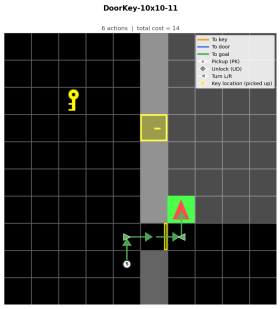
Fig. 10: Environments 5–8. Envs 5 and 6 achieve cost 25; Env 7 costs 26 (one extra action due to door state). Env 8 incurs cost 46, requiring key pickup and door interaction before reaching the goal.



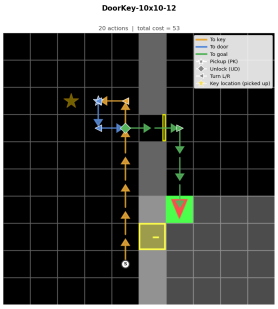
(a) Env 9 — cost 14



(b) Env 10 — cost 32

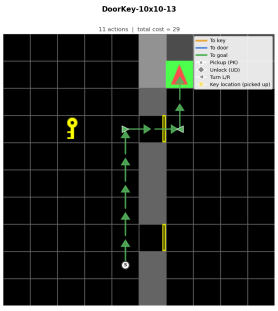


(c) Env 11 — cost 14

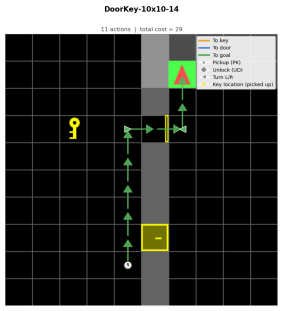


(d) Env 12 — cost 53

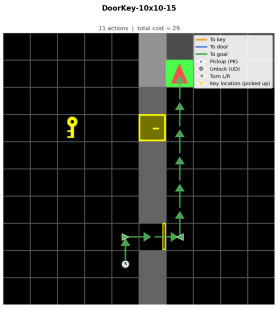
Fig. 11: Environments 9–12. Envs 9 and 11 achieve cost 14, the minimum across all Part B environments, because the goal is near the start position and is reachable via an open door with no key interaction. Env 12 costs 53, the near-maximum, requiring key pickup, door unlocking, and navigation to the farthest goal position.



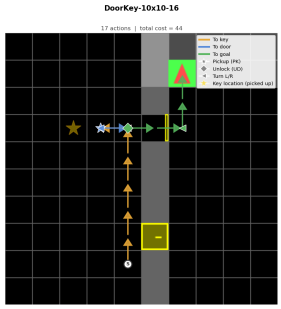
(a) Env 13 — cost 29



(b) Env 14 — cost 29

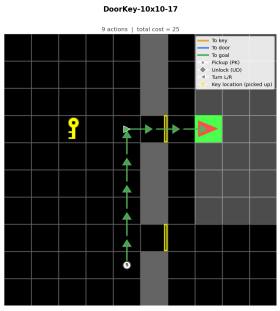


(c) Env 15 — cost 29

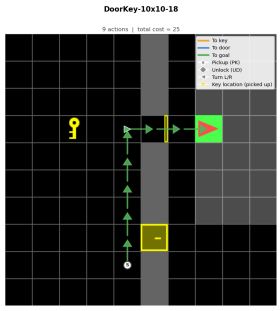


(d) Env 16 — cost 44

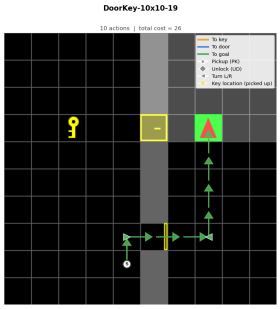
Fig. 12: Environments 13–16. Envs 13–15 all achieve cost 29 despite differing door states, showing that the universal policy routes identically when the door state does not change the minimum-cost path. Env 16 incurs cost 44 due to a locked door on the direct route to the goal.



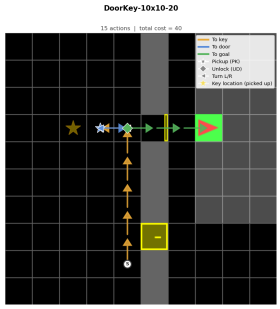
(a) Env 17 — cost 25



(b) Env 18 — cost 25



(c) Env 19 — cost 26



(d) Env 20 — cost 40

Fig. 13: Environments 17–20. The cost pattern mirrors group 5–8, reflecting the same key/goal layout with a different door-state combination. The unit difference between envs 17/18 and env 19 is attributable to a door that must be unlocked en route.

VI. Conclusion

This report formulated the Door-Key navigation problem as a deterministic, undiscounted MDP and solved it via synchronous value iteration. For the Known Map scenario, independent optimal policies were computed for seven environments, with convergence in 12–25 iterations and canonical costs ranging from 13 to 54 depending on grid size and door placement. For the Random Map scenario, a single DP solve over a 16,704-state extended space produced a universal policy valid for all 36 configurations without recomputation, converging in 25 iterations with costs between 14 and 53.

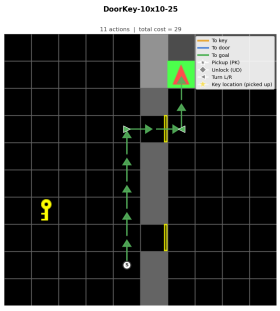
The key design decision enabling the universal policy, embedding the key index κ, goal index ϕ, and both door states into the state tuple, scales the problem by a factor of 18 over a single-configuration solve while eliminating the need for any runtime replanning. The results confirm that strictly positive stage costs and an absorbing goal state are sufficient to guarantee convergence, and that the extracted greedy policy is globally optimal in every tested environment.

Acknowledgment

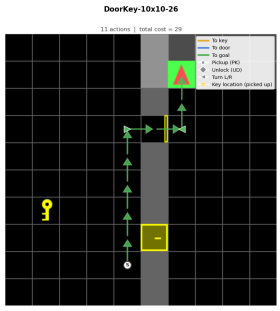
- Anthropic Claude-4.6 Sonnet - For Latex boilerplate code

VII. Appendix

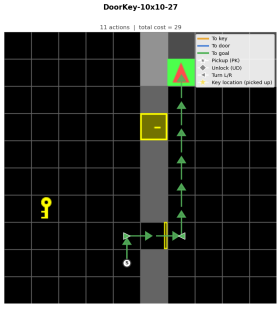
Below are rest of the trajectories in Random Map:



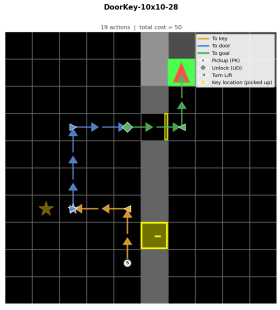
(a) Env 25 — cost 29



(b) Env 26 — cost 29

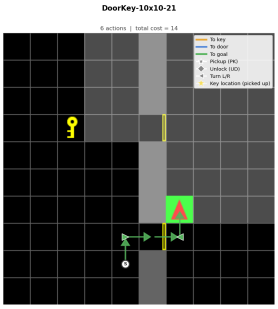


(c) Env 27 — cost 29

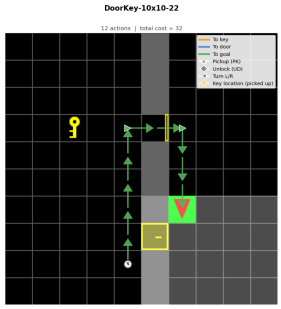


(d) Env 28 — cost 50

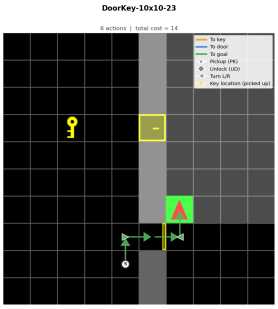
Fig. 15: Environments 25–28. Identical to the pattern in group 1–4 for this key/goal layout: three configurations at cost 29 and one at cost 50. The high-cost environment requires the agent to move across full grid width after unlocking the door.



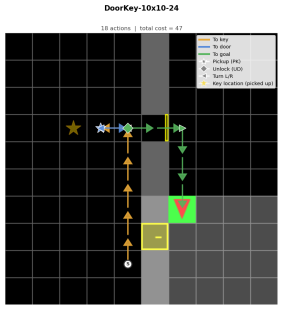
(a) Env 21 — cost 14



(b) Env 22 — cost 32

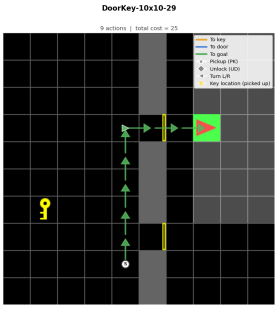


(c) Env 23 — cost 14

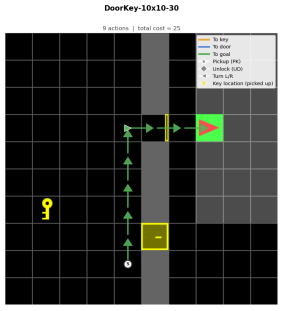


(d) Env 24 — cost 47

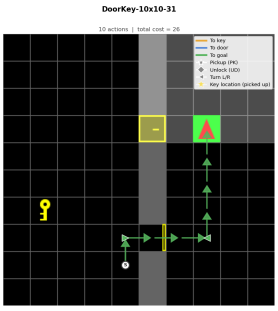
Fig. 14: Environments 21–24. The cost-14 configurations (envs 21 and 23) again correspond to open-door paths. Env 24 reaches cost 47 via a key-door sequence before navigating to the goal.



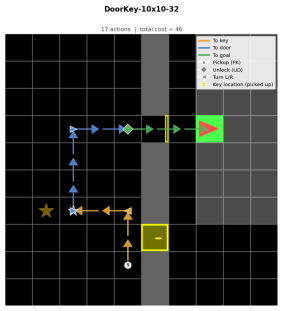
(a) Env 29 — cost 25



(b) Env 30 — cost 25

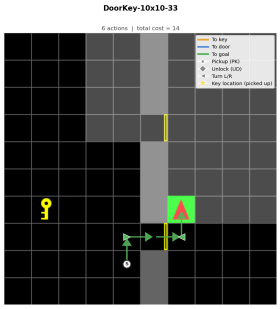


(c) Env 31 — cost 26

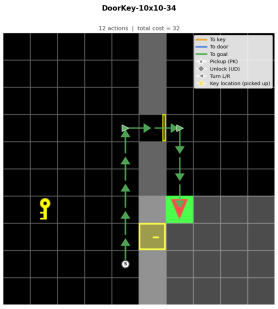


(d) Env 32 — cost 46

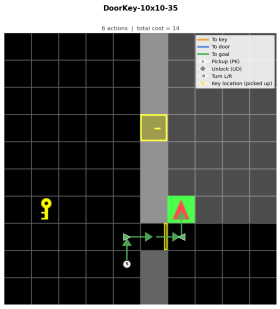
Fig. 16: Environments 29–32. Cost pattern mirrors groups 5–8 and 17–20 for this key/goal layout under the final door-state combination, confirming consistent policy behaviour across all four door configurations.



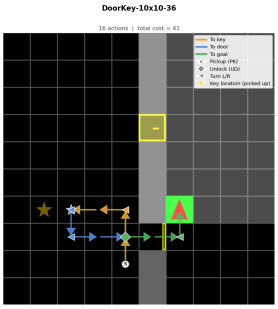
(a) Env 33 — cost 14



(b) Env 34 — cost 32



(c) Env 35 — cost 14



(d) Env 36 — cost 41

Fig. 17: Environments 33–36. The final group again exhibits the minimum-cost (14) configurations alongside higher-cost locked-door variants, closing the full enumeration of all 36 configurations.